INTRODUCTION TO MULTI-AGENT SYSTEMS

Beyond the Chatbot: Orchestrating AI Agents

Presented by Anish (23BCS016), Vishal (23BCS099)

The Core Problem

LLMs are incredibly intelligent text-predictors, but by default, they are trapped in a digital box.

If the Large Language Model is the Brain, LangChain provides the arms, legs, and memory needed to interact with the real world.

THE CORE ANALOGY

The Anatomy of an Agent

When we construct a Multi-Agent system, we assemble discrete "body parts" using LangChain as the central nervous system:

- **The Brain (LLM):** The core logic engine that interprets the goal and decides the next move.
- **The Persona (System Prompt):** Dictates the agent's behavior (e.g., "You are a harsh code reviewer").
- **The Hands (Tools):** The specific API endpoints the agent is allowed to trigger.
- **The Memory (Vector Store):** The database where the agent logs its previous actions and retrieves past context.

Unlocking the Model

The Trapped LLM (Default)

A standard LLM interacting via a raw API or chat interface has severe limitations:

- **Amnesia:** It forgets previous queries instantly.
- **Blindness:** It cannot search the live internet or read your company's private databases.
- **Paralysis:** It can write Python code, but it cannot actually *execute* it to test if it works.

The LangChain Solution

LangChain acts as a middle-layer framework that hooks the LLM up to external context and capabilities:

- **Persistent Memory:** Stores chat history and injects it into new prompts.
- **Data Connectors:** Reads private PDFs, Notion workspaces, and SQL databases.
- **Actionable Tools:** Allows the model to trigger APIs, send emails, or run code.

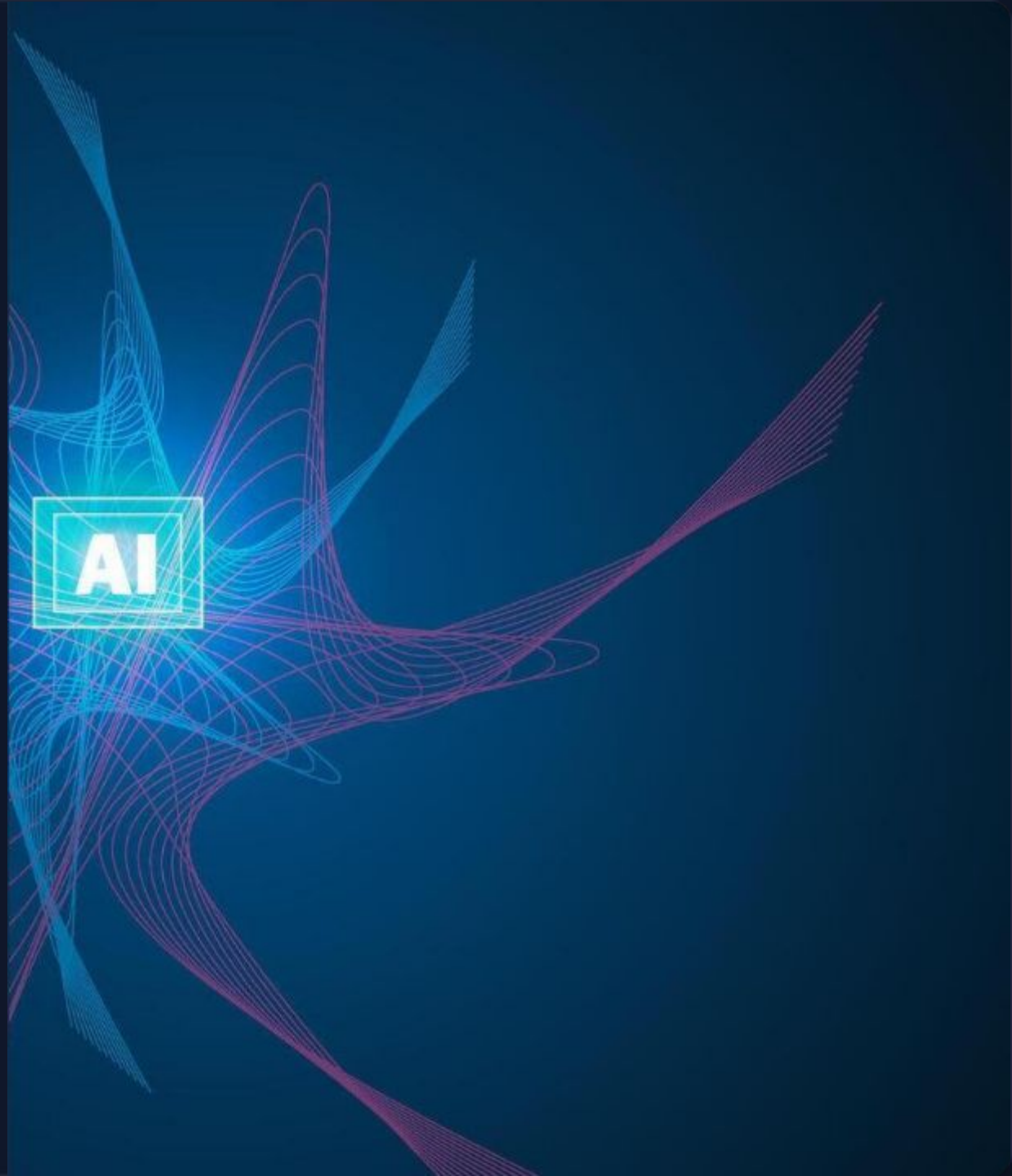
What is LangChain?

The Industry-Standard Framework

Created by Harrison Chase in 2022, LangChain is an open-source framework (available in Python and JS/TS) specifically designed to simplify the creation of generative AI applications.

It provides standardized, pre-built coding components that snap together like Lego bricks.

Instead of writing complex, custom API wrappers, memory management systems, and text parsers from scratch, developers use LangChain's unified syntax to build robust AI logic rapidly.



Why Use a Framework?

10x

Faster Development Time

LangChain vs. Manual Orchestration

Building an agent orchestrator manually is notoriously difficult. You have to write code to parse raw, unpredictable string outputs from the LLM, manage strict API rate limits, and write complex "while-loops" for the agent's reasoning process. LangChain eliminates this boilerplate. It provides built-in cognitive loops, robust error handling, automatic JSON parsing, and fallback models out-of-the-box. What takes 500 lines of custom code can be executed in just 5 lines of LangChain code.

The Building Blocks (Part 1)

Models & I/O

LangChain is "model-agnostic." It provides a universal interface. You can write your agent logic once, and effortlessly swap the underlying brain from OpenAI's GPT-4 to Google's Gemini, or to an open-source local model like Llama 3.

Prompt Templates

Hardcoding text prompts is messy. Templates allow you to dynamically inject variables (like the user's name, the current date, or retrieved data) into instructions before sending them to the language model.

Chains

The core concept. A "Chain" links modular components together. For example: taking user input → formatting the prompt template → calling the LLM → parsing the text output into a clean JSON object.

The Building Blocks (Part 2)

Retrieval (RAG)

Components for loading documents, chunking text, generating embeddings, and querying Vector Databases. This is how you give an agent access to a massive corporate wiki without exceeding the LLM's token limit.

Tools

The "Hands" of the operation. Tools are functions you grant the model permission to use. This can be a Web Scraper, a Wikipedia search, a Python terminal, or an API call to a smart home device.

Agents

The pinnacle of LangChain. Unlike rigid Chains, Agents use the LLM as a reasoning engine to dynamically decide *which* tools to use, and in *what order*, based on the user's initial prompt.

How an Agent "Thinks" (The ReAct Flow)

1. Observe

The Agent receives the user's complex prompt and evaluates the specific tools it has available in its toolkit.

2. Reason

The LLM formulates a plan. *"I need the current weather. I will use the Weather API Tool for New York."*

3. Act

LangChain executes the chosen tool. It pauses the LLM, makes the API call, and retrieves the raw data.

4. Finalize

The observation is passed back to the LLM. It analyzes the data and formats the final natural language answer.

Real-World Multi-Agent Apps

Automated Extraction

Agents reading unstructured emails and invoices, reasoning about the content, and formatting it into strict JSON payloads for databases.

DevOps Automation

A "Planner Agent" breaking down a user request, passing tasks to a "Coder Agent" to write scripts, and a "QA Agent" to test them autonomously.

Cyber Security Red-Teaming

Teams of agents actively probing a company's network, running simulated attacks, and documenting vulnerabilities without human oversight.

Live Demonstration

Moving from theory to code: Building a tool-equipped AI Agent using LangChain in real-time.

Ready for the Code?

Any final questions on the LangChain framework before we jump into the terminal?